Feuille d'Acquisition des Données (DAS) - pili-pos-android

Plateforme : Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google)

Conformément au §II.C du Manuel de certification SCE 2.0, ce document décrit **où** les données fiscales et applicatives sont stockées, **comment** y accéder, **combien de temps** elles sont conservées et **comment** elles peuvent être exportées pour un contrôle SPF Finances.

1. Sources de données

1.1 Base de données fiscale principale

Élément	Valeur
Moteur	SQLite 3.x (bundled via `rusqlite`)
Mode	WAL (Write-Ahead Logging)
Chemin (production)	`pos.db`, dans le répertoire `files/engine/` de la mémoire interne privée de l'application (`PLATEFORME_ANDROID_FR.md` §5), jamais sur une carte SD
Encodage	UTF-8
Fichiers annexes WAL	`pos.db-wal`, `pos.db-shm` (mêmes répertoire et accès)
Fichier de verrou	`engine.lock` (même répertoire) : fichier vide, qui ne contient aucune donnée ; le moteur y prend un verrou exclusif à l'ouverture, ce qui empêche deux instances d'écrire dans la même base
Propriétaire	L'UID unique de l'application (bac à sable d'Android)
Accès	Réservé à l'application : ni une autre application, ni l'opérateur ne lisent le fichier (`SECURITE_BD_FR.md` §3.1)

Schéma complet : voir `SCHEMA_BD_FR.md`.

1.2 Sauvegardes automatiques de la base

Élément	Valeur
Déclencheur	Après chaque rapport Z, juste après son archivage (clôture journalière fiscale)
Mécanisme	API SQLite Online Backup (cohérent en WAL)
Répertoire	`files/engine/backups/`
Format de nom	`pos-YYYYMMDD-HHMMSS.db`
Format	Fichier SQLite identique au fichier vivant
Accès	Ceux du stockage privé de l'application (§1.1)
Rétention	Les 7 plus récentes : après chaque Z, le moteur supprime les plus anciennes (décision du producteur du 2026-10-10 ; commit dans l'étiquette de gel : `4ff25b110c975d497f548538a08d81387b504b25` (commit « feat(engine): keep only the 7 newest database backups » du 2026-10-10, contenu dans l'étiquette de gel android-v1.0.0-rc3)). `fiscal_events` n'est jamais supprimé (§3).

Implémentation : `crates/pos-engine/src/commands/report.rs::backup_database_impl`.

1.3 Journaux applicatifs (logs structurés)

Élément	Valeur
Bibliothèque	`tracing` + `tracing-appender`
Format	Texte (une ligne par message, sans couleurs ; `tracing_subscriber::fmt`), recopié aussi dans logcat sous le tag `pili-pos`
Chemin (production)	`files/logs/pili-pos.log.YYYY-MM-DD`
Rotation	Quotidienne ; les 90 fichiers les plus récents sont conservés
Niveaux	TRACE, DEBUG, INFO (défaut), WARN, ERROR
Configuration runtime	Filtre `info`, fixé par l'application ; non modifiable par l'opérateur

Les logs ne sont **pas une source d'information fiscale** - ils ne contiennent pas d'événements N/P/T/C/I/F/S/R, uniquement des événements applicatifs (démarrage, erreurs, alertes imprimante). Aucun INSZ n'y est écrit (§6). Les données fiscales authoritatives sont dans `pos.db::fiscal_events`.

1.4 Réponses brutes du FDM (audit)

Stockées **intégralement** dans la colonne `fiscal_events.fdm_response_json` (type TEXT, JSON sérialisé). Chaque ligne contient la réponse complète du FDM (incluant la signature ECDSA brute, l'URL de vérification, le QR data, les éventuelles informations/avertissements).

Ceci permet à un contrôleur de comparer chaque événement passé, signature ECDSA comprise, à l'enregistrement que le FDM a transmis au SPF Finances. Avant d'archiver une réponse, le POS a contrôlé qu'elle répond bien à sa requête (`crates/pos-fiscal/src/response_check.rs`).

1.5 Ticket numérique par e-mail (§III.H)

Deux tables de `pos.db`, lisibles comme le reste de la base (§2 ci-dessous) :

Table	Contenu	Ce qu'un contrôleur peut y lire
`ticket_deliveries`	Une ligne par ticket dont le support demandé incluait l'e-mail (`DIGITAL`/`PAPER_DIGITAL`) : support, texte du ticket envoyé, URL de vérification, statut d'envoi, nombre de tentatives, dernière erreur.	L'adresse en clair (`recipient`) n'y figure que pendant les envois/réessais en cours, et au plus 72 h (purge RGPD, cf. §6 ci-dessous). L'indice masqué (`recipient_hint`, ex. `j***@e***.be`) reste consultable indéfiniment.
`original_prints`	Une ligne par événement fiscal dont l'original a été imprimé, avec l'horodatage de l'impression.	Sert à vérifier qu'aucun original n'a été imprimé deux fois (POS spec v2.2 §8.1) - toute reproduction ultérieure porte un événement C séparé dans `fiscal_events`.

Voir `TICKET_NUMERIQUE_FR.md` pour le détail du cycle de vie de `ticket_deliveries` et `SCHEMA_BD_FR.md` §3.6-3.7 pour le schéma complet.

1.6 Données de configuration

Élément	Valeur
Fichier de configuration	`pos.json`, dans `provisioning/bundle/`, partie du lot de configuration signé importé au provisionnement
Contenu	identifiants POS (posId, terminalId), URL FDM, adresse imprimante, infos restaurant, mode formation
Format	JSON
Identifiant de l'appareil (deviceId)	Ne vient pas de `pos.json` : il est lu dans le numéro de série de l'appareil à l'import et à chaque démarrage, et un identifiant écrit dans `pos.json` n'est pas pris en compte (`PLATEFORME_ANDROID_FR.md` §4)
Modifiable par l'opérateur	Non : le lot est vérifié par signature à l'import et ne change que par l'import d'un nouveau lot
Source de vérité runtime	Chargé à l'ouverture du moteur (`Engine::open`) dans `AppState.config`

1.7 Exports nocturnes (répertoire local `exports`, facultatif)

Seulement si la section `export` de `pos.json` est renseignée (absente par défaut : rien n'est alors créé). Après chaque rapport Z réel, le POS prépare dans le sous-répertoire `exports` de son répertoire de données (`files/engine/exports/`), jamais sur une carte SD ni un support amovible, un répertoire par rapport Z :

Élément	Valeur
Répertoire	`exports/<date AAAAMMJJ>-z<numéro du Z sur 6 chiffres>/`
`pos-<identifiant>.db.age`	Copie cohérente complète de `pos.db`, **chiffrée** (age, X25519) pour le propriétaire de l'établissement et, en option, le producteur. Illisible sans leur clé privée, qui n'est jamais sur la caisse.
`stats-<identifiant>.sqlite`	Copie statistique nettoyée de la période close : tickets, lignes, TVA, paiements, remises, Z ; opérateurs sous pseudonyme ; ni INSZ, ni nom, ni PIN, ni donnée de client. Description des tables : `EXPORT_NOCTURNE_FR.md` §6.
`manifest-<identifiant>.json`	Identité de l'installation, numéro du Z, noms, tailles et empreintes SHA-256 des deux autres fichiers.
Fichiers de suivi	`cursor.json`, `status.json`, `state.json` de chaque export (état d'envoi, sans secret) et `pseudonym.key` (clé des pseudonymes, mode 0600, jamais envoyée).
Propriétaire, accès	Ceux de l'application ; aucune lecture par l'opérateur.

Ces fichiers sont des **copies** : la source de vérité reste `pos.db::fiscal_events` (§1.1). Ils n'ajoutent aucune méthode d'accès pour le contrôleur (§2.2) : la sauvegarde étant chiffrée pour le propriétaire, le contrôleur lit toujours la base elle-même. Détail : `EXPORT_NOCTURNE_FR.md`.

1.8 Badges du personnel (option, désactivée par défaut)

Élément	Valeur
Tables	`badge_settings` (réglage unique), `staff_badges` (liens badge / employé, jamais supprimés), `badge_log` (liens, retraits, réglages ; ajout seul), `badge_scans` (passages ; ajout seul), dans `pos.db` (migration 70)
Donnée du badge	Une empreinte HMAC-SHA256 (64 caractères hexadécimaux) ; **jamais le numéro du badge**, ni en base, ni dans les journaux applicatifs, ni dans un événement fiscal ; les journaux portent une référence de 12 caractères de l'empreinte
Clé de l'empreinte	Fichier `badge.key` dans `files/engine/` : 32 octets aléatoires, mode 0600, créé une fois sur l'appareil. Hors de `pos.db`, donc hors des sauvegardes (§1.2) et des exports ; elle ne quitte jamais l'appareil
Données personnelles associées	INSZ de l'employé, INSZ du Gérant qui a lié ou retiré (`staff_badges.user_insz`, `linked_by`, `revoked_by`, `badge_log`, `badge_scans`)
Finalité	Enregistrement de la présence (événement S) et connexion à la caisse, rien d'autre ; aucune donnée biométrique

1.6 ter Fonctions du propriétaire (réglages, alertes, allergènes, temps, bons cadeaux)

Tables de `pos.db` lisibles comme le reste de la base (§2 ci-dessous), migrations 60 à 63 (*SCHEMA_BD_FR.md* §2.9) :

Élément	Valeur
Réglages	`owner_settings` (valeur courante de chaque réglage, JSON) et `owner_settings_log` (chaque enregistrement : qui, quand, valeur ; ajout seul)
Alertes	`owner_alert_log` (chaque message : situation, transition, détail sans donnée personnelle, résultat, code HTTP, heure ; ajout seul). Seul flux sortant : un court message HTTPS vers `api.telegram.org`, **uniquement** si le propriétaire a activé la fonction et fourni un jeton ; le jeton est un fichier, jamais en base
Allergènes	`menu_item_allergens`, `modifier_allergens`, `allergen_declarations`, `allergen_change_log` (ajout seul) ; information d'affichage, aucune donnée personnelle
Service en plusieurs temps	`order_line_courses`, `order_course_releases`, `kitchen_slip_entries` ; donnée de cuisine, jamais dans une demande signée
Bons cadeaux	`gift_vouchers`, `gift_voucher_movements`, `gift_voucher_prints` (ajout seul) ; le code complet d'un bon n'est imprimé que sur le bon
Données personnelles	Aucune donnée de client ni d'employé dans ces tables, hors l'INSZ du Gérant qui a réglé une fonction ou réimprimé un bon (`changed_by`, `printed_by`)

2. Accessibilité

2.1 Sur la machine du commerçant

- Pas de console, de shell, de SSH ni de compte de contrôle sur le terminal (`SECURITE_BD_FR.md` §3.3 et §3.4).
- L'opérateur ne dispose d'aucune commande shell ni d'explorateur de fichiers : l'application démarre en plein écran, en mode kiosque.
- Le transfert de fichiers par USB est interdit et le débogage USB est coupé dans une version de production.
- Les données sont dans le stockage privé de l'application, hors de portée de toute autre application (§1.1).

2.2 Accès pour audit / contrôle SPF Finances

Une seule méthode : **l'export SPF, lancé par le Gérant sur une clé USB ou une carte SD**. Le texte de référence est `SECURITE_BD_FR.md` §8 (droits de l'opérateur, étapes, contenu du dossier exporté, vérification par le contrôleur, absence d'autre voie). L'export reprend les sources du §1 ainsi :

Source	Dans l'export
Base fiscale (§1.1)	`pos.db`, copie fraîche et cohérente (API Online Backup de SQLite)
Sauvegardes (§1.2)	`backups/` (les 7 plus récentes, comme la rétention du §3) et `backups.txt`
Journaux (§1.3)	`logs/`
Identité (§1.6)	`identity.json` : posId, terminalId, deviceId, version et empreintes de l'application
Journal de la plateforme	`audit/`
Exports nocturnes (§1.7)	Non repris : ce sont des copies

Il n'y a ni console, ni disque que l'opérateur puisse extraire ; la fonction `backup_database_impl` est celle que l'export appelle ; l'interrogation SQL de la copie est décrite au §2.3.

2.3 Format de sortie de l'export

Une fois la base copiée, les données s'interrogent via SQL standard :

```sql
-- Toutes les ventes (Événement N) d'une journée donnée
SELECT id, sequence_number, order_id, user_insz, created_at,
       fdm_id, event_label, short_signature
FROM fiscal_events
WHERE event_type = 'N'
  AND DATE(created_at) = '2026-10-03'
ORDER BY sequence_number;

-- Détail d'un événement spécifique (request + response complets)
SELECT event_data_json, fdm_response_json
FROM fiscal_events
WHERE id = 42;

-- Récap caisse d'une journée (totaux par mode de paiement)
SELECT method, SUM(amount_cents) / 100.0 AS total_eur, COUNT(*) AS nb
FROM payments
WHERE DATE(created_at) = '2026-10-03'
GROUP BY method;
```

3. Rétention

Donnée	Durée de rétention	Justification
`fiscal_events`	Indéfinie	Code Belge des Impôts - 7 ans minimum, jamais purgé
Sauvegardes (`pos-*.db`)	Les 7 plus récentes (§1.2), les plus anciennes sont supprimées après chaque Z	Chaque copie est une copie complète de `pos.db` : le journal fiscal n'en est jamais supprimé et contient tout ce que contenaient les copies supprimées
Logs applicatifs	90 fichiers journaliers, puis suppression du plus ancien	Permettre un audit de comportement applicatif a posteriori ; non fiscaux
Journal de la plateforme (`audit.jsonl`, répertoire `platform/`)	Rotation à 1 Mio, 5 fichiers conservés	Trace des règles appliquées, exports et mises à jour ; non fiscal
`payments`, `orders`, `order_lines`	Indéfinie	Données de support nécessaires à la reconstruction d'un ticket
Translations, menu_items, modifier_groups, modifiers	Indéfinie	Données de référence. Le menu est mis à jour en place (y compris par la synchronisation du catalogue, qui désactive sans supprimer) ; l'historique fiscal n'en dépend pas : nom, prix et TVA sont figés dans `order_lines` et `order_entries`
`orders.raw_hubrise_json`, `hubrise_events` (commandes en ligne)	Indéfinie, sans purge automatique	Contiennent des données de clients (§6)
`staff_badges`, `badge_log`, `badge_scans` (badges du personnel)	Indéfinie, jamais supprimées (triggers), avec les données de présence	Audit des accès et de la présence ; l'empreinte est inutilisable sans la clé de l'appareil (§1.8) ; un retrait date la ligne sans l'effacer
Exports nocturnes locaux (§1.7), s'ils sont activés	`keep_days` jours (14 par défaut, de 1 à 365), envoyés ou non, puis supprimés automatiquement	Simples copies, ni sources ni archives : la conservation légale est celle de `fiscal_events` (§1.1), que cette suppression ne touche pas. Un export jamais livré qui est supprimé est compté (« Abandonnés ») et journalisé. La durée de conservation chez le récepteur est décidée par le propriétaire de l'établissement (`PROFIL_ENTREPRISE_FR.md` §6.3).

Aucune purge n'est implémentée sur les données légales : le journal fiscal de `pos.db` n'est jamais supprimé. Le passage à un modèle de purge légalement nécessitera une modification certifiée soumise au SPF Finances. Les seules suppressions automatiques portent sur des copies et des journaux techniques (sauvegardes au-delà des 7 plus récentes, journaux au-delà de 90 fichiers, exports locaux au-delà de `keep_days`).

4. Protection contre la suppression / modification

- Cf. `DECLARATION_FONCTIONS_INTERDITES_FR.md` §1 et §2.

- La table fiscal_events ne fait l'objet d'aucun DELETE/UPDATE dans le code applicatif (vérifié par grep), et des triggers SQLite refusent toute suppression ou réécriture, même par un accès direct à la base (fiscal_events_no_delete, fiscal_events_no_rewrite ; idem z_reports_*, order_entries_*, pending_signs_*, booking_periods_*, staff_badges_* (aucune suppression, retrait seul), badge_log_*, badge_scans_* ; liste dans SCHEMA_ER_FR.md §4).
- Les sauvegardes sont stockées en fichiers indépendants, jamais écrasés (le nom inclut le timestamp à la seconde) ; seules les plus anciennes, au-delà des 7 conservées, sont supprimées par le moteur (§3).
- L'opérateur n'a ni shell ni explorateur de fichiers pour supprimer un fichier de données.

5. Intégrité

- WAL (Write-Ahead Logging) garantit l'atomicité crash-safe.
- Chaque opération fiscale est signée sous un verrou unique, puis archivée dans une transaction SQLite qui inclut compteur + archive + (le cas échéant) fermeture commande + clôture de la ligne pending_signs. Cf. STRATEGIE_COMPTEURS_FR.md §5.
- La signature ECDSA secp256r1 stockée dans fiscal_events.fdm_response_json est celle que le FDM a calculée et transmise au SPF Finances : une ligne modifiée rétroactivement ne correspondrait plus à l'événement signé détenu par le FDM et le SPF.
- Empreintes SHA-1 de l'APK signé pili-pos-android et de sa bibliothèque libpos_ffi.so (le moteur) consignées dans le formulaire de demande de certification : À effectuer (EV-01 et EV-02 : valeurs de la version 1.0.0). Le terminal déployé les affiche (écran « À propos » et identity.json de l'export) ; ces valeurs viennent de l'application elle-même et ne sont pas une vérification indépendante (SECURITE_BD_FR.md §4.2).

6. Confidentialité (GDPR)

- INSZ du personnel : numéro national, donnée personnelle identifiante.
- INSZ stocké dans les colonnes users.insz, orders.user_insz, fiscal_events.user_insz, pending_signs.user_insz, staff_shifts.user_insz, staff_shifts.started_by, staff_shifts.ended_by, till_discounts.manager_insz, invoices.created_by, b2b_customers.updated_by, menu_item_sold_out.sold_out_log.by, menu_change_log.changed_by, webshop_orders.accepted_by, integration_switches.changed_by, integration_switch_log.changed_by, staff_badges.user_insz, badge_log.user_insz et badge_scans.user_insz, ainsi que dans les JSON fiscal_events.event_data_json (payload envoyé au FDM, obligation fiscale), pending_signs.request_body, pending_signs.archive_json et pending_signs.context_json, et z_reports.data_json (rapport par utilisateur). Les rapports imprimés par utilisateur (X/Z) portent aussi l'INSZ.
- Aucun INSZ n'est écrit dans les journaux applicatifs : les messages qui concernaient un opérateur (vente soldée sans encaissement) n'enregistrent que son rôle, et aucun filtre de masquage n'est nécessaire.
- Pas de transmission de l'INSZ hors du périmètre POS+FDM (la synchronisation HubRise n'envoie ni INSZ ni nom d'employé). Voir PROFIL_ENTREPRISE_FR.md §GDPR pour le détail des bases légales (article 6.1.c GDPR - obligation légale).
- Adresse e-mail du ticket numérique (§1.5) : donnée personnelle fournie volontairement par le client, à la demande de celui-ci (base légale : demande du client, cf. PROFIL_ENTREPRISE_FR.md §6). Conservée en clair au maximum 72 h (ticket_deliveries.recipient), effacée immédiatement dès l'envoi réussi ou le repli papier ; seul un indice masqué survit indéfiniment. Jamais journalisée en clair, jamais affichée dans l'écran "Tickets numériques" des Paramètres (list_ticket_deliveries n'expose que l'indice masqué). Sous-traité à Scaleway (Transactional Email, UE, région fr-par) pour l'envoi uniquement - aucun usage marketing.
- Données de clients des commandes en ligne (si l'intégration HubRise est activée pour l'installation, désactivée par défaut) : orders.raw_hubrise_json et hubrise_events.payload_json conservent la commande telle que reçue, avec nom, téléphone et adresse ; le ticket client non fiscal les imprime ; les ventes de la caisse sont renvoyées à HubRise par hubrise_outbox (lignes, totaux, paiements, sans données de client). Aucune purge automatique. Le jeton d'accès HubRise (hubrise_connection.access_token) est stocké en clair dans pos.db : il est présent dans les sauvegardes et dans la copie de pos.db de l'export SPF. Détail dans SECURITE_BD_FR.md §6.

- Export nocturne (§1.7), si la section `export` est renseignée : la sauvegarde complète contient toutes les données personnelles de `pos.db` (INSZ, données des clients des commandes en ligne), mais elle ne quitte la caisse que **chiffrée pour le responsable de traitement** ; le serveur de réception ne peut pas la lire. La copie statistique ne contient ni INSZ, ni nom, ni PIN, ni donnée de client : seulement des **pseudonymes** d'opérateurs, calculés avec une clé propre à l'installation qui n'est jamais envoyée. Un pseudonyme reste une donnée personnelle. Base légale, rôles et durées : `PROFIL_ENTREPRISE_FR.md` §6 ; détail : `EXPORT_NOCTURNE_FR.md` §5 et §6.
- Badges du personnel (§1.8, option désactivée par défaut) : le numéro du badge n'est jamais conservé, seulement son empreinte HMAC-SHA256 calculée avec une clé qui reste sur l'appareil (hors base, sauvegardes et exports) ; aucune donnée biométrique ; usage limité à la présence et à la connexion ; l'employé peut refuser le badge (`PROFIL_ENTREPRISE_FR.md` §6.1). Le journal des passages (`badge_scans`) conserve l'heure, l'INSZ et le résultat, jamais le numéro.

7. Procédure d'export à fournir au contrôleur

Lors d'un contrôle, le distributeur ou son représentant fournit :

1. Un export complet par le Gérant (§2.2) : `pos.db`, les sauvegardes conservées, `identity.json`.
2. Les journaux applicatifs couvrant la période contrôlée (`logs/` du même export) et le journal de la plateforme (`audit/`).
3. Les empreintes SHA-1 de l'APK et de `libpos_ffi.so` en cours d'exécution sur le terminal (écran « À propos », `identity.json`).

Le POS ne détient pas la clé publique du FDM : la vérification cryptographique des signatures se fait avec les données du FDM et du SPF Finances. Le contrôleur peut, à partir de l'export : - Comparer chaque événement (signature comprise) aux enregistrements du FDM et du SPF Finances. - Recalculer chaque shortSignature d'un événement N : Hex(SHA1(Base64Decode(digitalSignature))). - Reconstituer la séquence chronologique des événements. - Croiser avec les tickets imprimés (vérification QR code). - Repérer toute incohérence éventuelle entre l'archive locale et les enregistrements cloud SPF Finances.

8. Réseaux et appareils (section B de la feuille du SPF)

Réponse du producteur aux questions sur les réseaux et les appareils :

Question	Réponse
Connexion du POS au FDM	Réseau local privé, selon l'installation : GraphQL en HTTPS avec mTLS (port 443) ou en HTTP (port 80). Seul le FDM communique avec le SPF Finances. Le type de liaison du D3 Pro réel : À effectuer (EV-15 : liaison réseau du terminal réel).
Connexion du POS à l'imprimante	Réseau local, imprimante Epson TM-m30III, TCP 9100 (ESC/POS).
Le POS nécessite une connexion Internet	Non par défaut. Quatre sorties HTTPS (443) facultatives, chacune seulement quand elle est configurée ou activée : ticket numérique (`api.scaleway.com`), commandes en ligne (`api.hubrise.com`, `manager.hubrise.com`, par `hubrise_enabled`), envoi des factures B2B (`api.storecove.com`, section `peppol`), et **export nocturne** (point de réception propre à l'installation, section `export` de `pos.json`). Les racines TLS sont embarquées dans l'application. Aucun port n'écoute sur le terminal.

Question	Réponse
Connexion avec un logiciel interne (statistiques et sauvegarde)	Oui, facultative : l'export nocturne. Une requête HTTPS `PUT` par fichier, après chaque rapport Z réel, avec jeton porteur lu dans un fichier séparé et empreinte SHA-256 de chaque fichier. Contenu : une sauvegarde chiffrée de bout en bout (illisible pour le serveur de réception) et une copie statistique à opérateurs pseudonymisés. Elle n'est jamais nécessaire à une vente ni à un Z ; en son absence ou en cas de panne, la caisse fonctionne à l'identique. `EXPORT_NOCTURNE_FR.md`.
Connexion avec des logiciels externes	HubRise (commandes en ligne), coupé par défaut ; `COMMANDES_EN_LIGNE_FR.md`.
Accès distant	Aucun : pas de programme de prise de main à distance sur Android ; le support se fait sur site. Aucun port entrant.
Appareils supplémentaires	Aucun.